

Submission Worksheet

IT202-450-M2026 / module-07-worksheet-project-api-testing

Submission Data

Course	IT202-450-M2026
Assignment	Module 07 Worksheet: Project API Testing
Student	Michos C. (mrc82)
Status	submitted
Worksheet Progress	100%
Potential Grade	10.00/10.00 (100.00%)
Received Grade	0.00/10.00 (0.00%)
Updated	7/21/2026, 7:32:06 PM

Grading Link

<https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/module-07-worksheet-project-api-testing/grading/mrc82>

View Link

<https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/module-07-worksheet-project-api-testing/mrc82>

Instructions

Adapt the Module 7 API lesson patterns to the separate API selected for your project, then collect focused evidence that PHP can request, validate, and map that API safely. The Alpha Vantage stock/company API and the lesson test pages are practice examples; keeping or submitting those examples does not satisfy this homework. The StarCraft II API is also not acceptable since future lessons will use that in a guided project.

Assignment Setup

- Required API choice: the separate project API recorded in the class API-selection Google Sheet on Canvas
- Required branch: `Module07-Homework`
- Required starting branch: `qa`
- Required project folder: `public_html/project/`
- Required project API wrapper: `lib/project_api.php` (or equivalent)
- Required project API test page: `public_html/project/test_api.php` (or equivalent)
- Required cached sample: `lib/api_samples/project-api-sample.json` (or equivalent)
- Required deployed evidence target: working `qa` site before production promotion
- Lesson reference: [Working With API Data branch](#)
- Recommended order:
 - Read the whole worksheet and the chosen API's official documentation
 - Confirm the API choice, quota, authentication method, endpoint, and useful JSON keys
 - Check out `qa`, pull the latest version, and create the homework branch
 - Keep the shared API helpers from the lessons and create the project-specific wrapper, test page, and cached sample
 - Add the wrapper import to `lib/app.php` after the generic API helper import
 - Test cached behavior before spending additional live API quota
 - Test one intentional failure and one successful live response
 - Merge into `qa`, test the deployed behavior, then collect worksheet evidence

- Test one intentional failure and one successful live response
- Merge into `qa`, test the deployed behavior, then collect worksheet evidence

Git Workflow Checklist

1. Check out `qa` with `git checkout qa`.
2. Pull the latest `qa` branch with `git pull origin qa`.
3. Create the homework branch with `git checkout -b Module07-Homework`.
4. Create `lib/project_api.php`, `public_html/project/test_api.php`, and `lib/api_samples/project-api-sample.json` (or equivalent files).
5. Commit the project-specific API file structure before the final integration is complete.
6. Push the branch with `git push origin Module07-Homework`.
7. Open a pull request from `Module07-Homework` into `qa` and keep it open while you work.
8. Commit the working wrapper, response mapping, cached test, live test, and failure handling in focused pieces.
9. After all API tests pass locally, merge the homework pull request into `qa`.
10. Gather evidence from the deployed `qa` site and fill out the worksheet.
11. Export the worksheet PDF from the Learn Courses Platform.
12. Save the PDF in the repository under `docs/module07/`.
13. Add, commit, and push the PDF to the `Module07-Homework` branch.
 - `git add .`
 - `git commit -m "Add module 7 worksheet PDF"`
 - `git push origin Module07-Homework`
14. Upload the same PDF to Canvas.
15. Create and merge a pull request from `qa` into `prod` so the anticipated production URL becomes live.
16. Switch back to `qa` with `git checkout qa`.
17. Pull the latest `qa` branch with `git pull origin qa`.

Shared Requirements

Apply these requirements to the project-specific API integration:

- Use a different API from the Alpha Vantage lesson example and future StarCraft II API (both should be shown in the Google Sheet on Canvas)
- Use server-side PHP for any API request that requires a private key or secret
- Keep real API secrets out of GitHub, screenshots, browser output, and worksheet responses
- Store local values in ignored `lib/.env` content and configure the same environment variable names in both `qa` and `prod`
- Keep the reusable `lib/api_helper.php` and response-validation patterns from the lessons
- Load `lib/project_api.php` (or equivalent) through `lib/app.php`; do not require the wrapper separately on each project page
- Call the live API only after a deliberate form submission or other intentional action
- Validate configuration, transport, HTTP status, response body, JSON decoding, and expected JSON keys
- Map the chosen API response into project-friendly field names instead of displaying only the raw API response
- Identify at least three meaningful API fields that can become project table columns later
- Transform nested API data into useful values or a planned relational shape instead of planning one raw JSON/text database dump
- Keep a secret-free cached response that follows the same expected response shape as the live request
- Use user-friendly page messages and send technical details to `error_log()`
- Add comments with your UCID, date, and a brief summary of your project-specific code
- Use your own API, code (based on the lesson), cached response, deployed output, branch, pull request, and explanations for evidence
- Provide exactly the requested evidence; ask for help or explain a blocker instead of submitting placeholders

Evidence Format

Evidence Format

Most implementation sections use the same evidence pattern:

- Images:
 - Show the relevant code and matching browser output
 - Make sure UCID/date comments are visible in code evidence
 - Make sure the deployed `qa` URL is visible in browser evidence
 - Do not show private API keys, tokens, authorization headers, or secret values
 - Use multiple screenshots when one image would be hard to read
- URLs:
 - Provide direct GitHub file links from `Module07-Homework`
 - Provide the anticipated production URL for `public_html/project/test_api.php` (or equivalent)
 - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename
 - PHP page URLs must end in `.php`; zero credit for that URL entry if it does not
- Response:
 - Explain the logic in your own words
 - Describe the request, response checks, mapping, or failure decisions made by the code
 - Do not restate the prompt or list the requirements

Submission Check

- Confirm `Module07-Homework` has been pushed
- Confirm the pull request into `qa` exists
- Confirm meaningful setup, wrapper, cached-response, live-request, and failure-handling commits are visible
- Confirm `lib/.env` and real API secrets are not committed
- Confirm the chosen API is different from the Alpha Vantage lesson API
- Confirm both cached and live tests use the project-specific API response shape
- Confirm the deployed `qa` test page was reviewed before promotion
- Confirm the anticipated production URL uses the same path as the tested `qa` URL with `-qa` changed to `-prod`
- Confirm the worksheet PDF is committed under `docs/module07/` on the homework branch
- Upload the same worksheet PDF to Canvas

Submission Progress

100%

Section #1: (3 pts.) Api Choice Analysis

100%

- Goal: confirm that the selected API is practical and returns useful project data
- The selected API must differ from the Alpha Vantage lesson example

🔗 Task #1 (0.75 pts.) – API Documentation

100%

📄 Details

- Paste the official API documentation or marketplace documentation URL for the selected API
- Link to the specific API or endpoint documentation, not a general search page

URL #1

<https://nickandmortyapi.com/documentation> 🍷

Task #2 (0.75 pts.) – Quota And Limitations

100%

Details

- State the free quota, rate limit, cost boundary, and whether the limit is hard or soft
- Briefly explain any restriction that could affect the project

Response

The Rick and Morty API is free and public, with no API key or paid plan required. The documented rate limit is 100 requests per minute. I would treat this as a hard per-minute limit, so requests above that amount may be blocked until the next minute. This should not affect my project because the live API is only called after an intentional form submission, and the cached sample prevents unnecessary requests during testing.

Supports **bold**, *italic*, [links](url), `code`, etc.

426 chars

Task #3 (1.5 pts.) – Project Data Plan

100%

Details

- Describe the endpoint and project data that will be tested
- Name at least three useful API JSON keys and the project-friendly field or future database column each one will become
- Briefly note any nested data that will need transformation or a related table

Response

I will test the Rick and Morty character endpoint at `/api/character` by submitting a character name as the name query parameter. The response can return multiple matching character records. The API key id will become `api_id`, name will become `name`, status will become `status`, species will become `species`, gender will become `gender`, and image will become `image_url`.

The origin and location values are nested objects, so I map `origin.name` to `origin_name` and `location.name` to `location_name` instead of storing the full nested JSON. The episode field is an array of episode URLs and could later be stored using a related character-episode table rather than one raw text field.

Supports **bold**, *italic*, [links](url), `code`, etc.

670 chars

Section #2: (6 pts.) Verify Api Choice

100%

- Goal: show that the selected project API works through server-side PHP on the deployed qa site
- The Alpha Vantage lesson pages do not count as project API evidence

Task #1 (6 pts.) – Project API Test Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the relevant `public_html/project/test_api.php` (or equivalent) code and project API wrapper code as needed,

① Details

- Show the relevant `public_html/project/test_api.php` (or equivalent) code and project API wrapper code as needed, with UCID/date comments visible
- The code evidence should show the intentional choice between live and cached requests, response validation, and mapping for at least three useful API values
- Show the deployed `qa` test page after a successful live request with the complete URL and mapped values visible
- Do not show API keys, tokens, authorization headers, or other secret values



Test page showing UCID/date, live or cached selection, input validation, wrapper call, and safe error handling.



Wrapper showing cached/live requests, expected results validation, API response mapping, and safe handling of missing data.



Project wrapper mapping Rick and Morty API data, including nested origin and location values, into project-friendly fields.



Deployed QA page showing a successful live Rick search with the full QA URL and mapped character values visible.

🔗 Part 2: URLs 100%

① Details

- Paste the direct GitHub file link for `public_html/project/test_api.php` (or equivalent)
 - The GitHub URL must end in `.php`; zero credit for this URL if it does not
- Paste the anticipated production URL for the same test page
 - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename
 - The production URL must end in `.php`; zero credit for this URL if it does not

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Module07-Homework/public_html/project/test_api.php 🍏

URL #2

https://mrc82-it202-450-m2026-prod.onrender.com/project/test_api.php 🍏

🔗 Part 3: Response 100%

① Details

- Explain what action and input produced the successful live request
- Explain how the response is checked and how at least three API fields will be mapped for the project
- Briefly explain how the cached sample conserves quota and how request failures are handled safely

Response

I entered Rick in the character-name field and clicked Search Live API. The form submits intentionally through POST, and PHP calls the Rick and Morty character endpoint using the submitted name as a query parameter.

I entered Rick in the character-name field and clicked Search Live API. The form submits intentionally through POST, and PHP calls the Rick and Morty character endpoint using the submitted name as a query parameter.

The shared API helper checks whether the request succeeded, verifies the HTTP status and response body, decodes the JSON into a PHP array, and confirms that the expected results key contains an array. The project wrapper then maps API fields into cleaner project values. For example, id becomes api_id, name becomes name, status becomes status, species becomes species, origin.name becomes origin_name, and location.name becomes location_name.

The cached sample follows the same response structure as the live API, so it can test validation and mapping without repeatedly using the API. Missing results produce a user-friendly "No matching characters were found" message, while technical request details are written to error_log() instead of being shown to the user.

Supports **bold**, *italic*, [links](url), ``code``, etc.

984 chars

Section #3: (1 pt.) Misc

100%

Task #1 (0.33 pts.) – GitHub Details

100%

Details

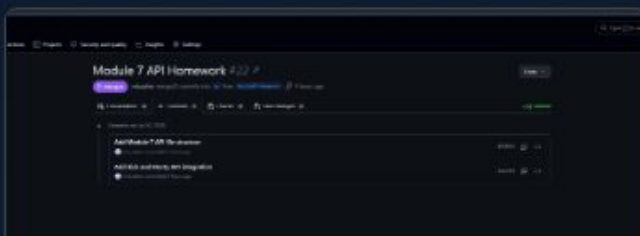
- Show the repository evidence for the Module 07 homework.

Combo #1

Part 1: Images 100%

Details

- Screenshot the Commits tab of the pull request.
- The screenshot should show at least the baseline commit and meaningful API integration commits.



Pull request Commits tab showing the baseline file-structure commit and the completed Rick and Morty API integration commit.

Part 2: URLs 100%

Details

- Paste the pull request URL.
 - The URL must end in `/pull/#`, where `#` is the pull request number; zero credit for this URL if it does not.

URL #1

<https://github.com/micopher/mrc82-IT202-450-M2026/pull/22> 🍏

Task #2 (0.33 pts.) – WakaTime Activity

100%

Details

- Visit the WakaTime dashboard.
- Open **Projects**.
- Find your repository.
- Screenshot the overall time near the repository name (top of page).
- Screenshot the file-level activity (near the bottom).
- The purpose is evidence of activity; the duration isn't tied to the grade.
- The visual graphs are not required.



WakaTime project overview showing the repository name and overall coding activity time.



WakaTime file-level activity showing time recorded across the Module 7 API project files.

Task #3 (0.33 pts.) – Reflection

100%

Details

- Answer each question with at least a few reasonable sentences.

Task #1 (0.11 pts.) – What did you learn during this assignment?

100%

Response

In this module, I learned how to use PHP to request data from an external API and turn the JASON response into cleaner project-friendly fields.

Supports **bold**, *italic*, [\[links\]\(url\)](#), ``code``, etc.

143 chars

Task #2 (0.11 pts.) – What was the easiest part of the assignment?

100%

Response

The easiest part was choosing the Rick and Morty API and testing the cached sample. Compared with the previous modules and Milestone 1, this assignment was pretty chill.

Supports **bold**, *italic*, [\[links\]\(url\)](#), ``code``, etc.

169 chars

Supports **bold**, *italic*, [links](url), `code`, etc.

169 chars

 Task #3 (0.11 pts.) – What was the hardest part of the assignment?

100%

Response

The hardest part was the coding because it took a long time to build and test everything correctly. I also had to troubleshoot the SSL certificate issue before the live API request worked.

Supports **bold**, *italic*, [links](url), `code`, etc.

188 chars

End of Assignment - submission has been recorded.